Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Новгородский государственный университет имени Ярослава Мудрого»
(НовГУ)
Политехнический институт

Кафедра информационных технологий и систем

ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ
программной системы
«Контур»

Выполнил:
студент группы 2093
Граванов М.А.
Проверил:
руководитель ВКР / преподаватель
_____

Великий Новгород
2026 г.

Содержание

1 C4-модель 3
1.1 Контекстная диаграмма 3
1.2 Диаграмма контейнеров 3
1.3 Диаграмма компонентов 3
2 Логическая архитектура 4
2.1 Состав модулей системы 4
2.2 Взаимодействие модулей 4
2.3 UML-диаграмма компонентов 4
3 Архитектура базы данных 5
3.1 Ключевые сущности 5
3.2 Связи и ограничения 5
3.3 ERD-диаграмма 5
4 Выбор паттернов проектирования 6
4.1 Паттерн Repository 6
4.2 Паттерн Adapter 6
4.3 Паттерн Strategy 6
Приложение А 8

1 C4-модель

Для описания архитектуры системы используется C4-модель. Сами диаграммы вынесены в приложение А, а в разделе приводятся только краткие ссылки на соответствующие представления.

1.1 Контекстная диаграмма

Контекстная диаграмма приведена в приложении А на рисунке А1. На ней показаны основные категории пользователей, агентный контур и внешние системы, с которыми взаимодействует система.

1.2 Диаграмма контейнеров

Диаграмма контейнеров приведена в приложении А на рисунке А2. На ней отражены основные технологические контейнеры решения, а также используемые протоколы взаимодействия между ними.

1.3 Диаграмма компонентов

Диаграмма компонентов приведена в приложении А на рисунке А3. Для детализации выбран основной API-контур, внутри которого показаны ключевые модули прикладной логики и инфраструктурные адаптеры.

2 Логическая архитектура

В разделе приводится логическое разбиение системы на основные прикладные модули и показываются интерфейсы их взаимодействия.

2.1 Состав модулей системы

На уровне логической архитектуры выделены четыре основных модуля: пользовательский интерфейс, прикладная бизнес-логика, сервис уведомлений и слой данных.

2.2 Взаимодействие модулей

Пользовательский интерфейс обращается к интерфейсу прикладных сервисов. Прикладная бизнес-логика координирует работу с задачами, документами и заметками, использует сервис уведомлений и обращается к слою данных через абстракции репозиториев.

2.3 UML-диаграмма компонентов

UML-диаграмма компонентов логической архитектуры приведена в приложении А. На ней показаны основные программные модули системы и интерфейсы взаимодействия между пользовательским интерфейсом, прикладной логикой, сервисом уведомлений и слоем данных.

3 Архитектура базы данных

В разделе рассматривается структура данных системы, состав основных сущностей и связи между ними. Модель данных построена в нормализованном виде и ориентирована на соблюдение требований не ниже третьей нормальной формы.

3.1 Ключевые сущности

Ключевые сущности базы данных можно разделить на несколько групп: организационные сущности (‘Workspace‘, ‘WorkspaceMember‘, ‘Project‘), сущности управления работами (‘TaskGroup‘, ‘Task‘, ‘TaskDependency‘), сущности базы знаний (‘Document‘, ‘Asset‘, ‘Note‘), а также сущности агентного и интеграционного контуров (‘Agent‘, ‘AgentCredential‘, ‘AgentSession‘, ‘IntegrationConnection‘, ‘AuditEvent‘).

3.2 Связи и ограничения

В модели используются связи типов «один ко многим», «один к одному или многим» и «многие ко многим», последняя из которых реализуется через промежуточную сущность ‘AgentSessionTask‘. Для обеспечения целостности применяются внешние ключи, а повторяющиеся и составные зависимости вынесены в отдельные таблицы, например ‘TaskDependency‘, ‘AgentCredential‘ и ‘AuditEvent‘.

3.3 ERD-диаграмма

ERD-диаграмма базы данных приведена в приложении А. На ней показаны основные сущности, их атрибуты, а также кардинальности и типы связей между таблицами.

4 Выбор паттернов проектирования

При проектировании системы выбраны три паттерна: Repository, Adapter и Strategy. В контексте реализации на Rust UML-диаграммы классов используются как средство описания структур, трейтов и сервисных компонентов, выполняющих роли классов и интерфейсов.

4.1 Паттерн Repository

Паттерн Repository применяется для изоляции прикладной логики от деталей хранения данных. Бизнес-модули работают с абстракциями репозиториев, а конкретные реализации инкапсулируют обращение к PostgreSQL и поисковым индексам.

Примером применения является работа с задачами, документами и заметками, где прикладные сервисы используют интерфейсы репозиториев вместо прямых SQL-запросов. UML-диаграмма класса для этого паттерна приведена в приложении А.

4.2 Паттерн Adapter

Паттерн Adapter используется для подключения внешних инфраструктурных зависимостей через внутренние абстракции системы. Это позволяет не связывать доменную модель с конкретным API провайдера и упрощает замену внешнего компонента без переработки прикладной логики.

Основной пример применения в системе — абстракция AssetStorage, скрывающая различия между локальным файловым хранением и S3-compatible backend. UML-диаграмма класса для этого паттерна приведена в приложении А.

4.3 Паттерн Strategy

Паттерн Strategy применяется там, где система должна поддерживать несколько взаимозаменяемых вариантов поведения. В архитектуре системы такой подход целесообразен для выбора механизма аутентификации человека

в зависимости от режима работы и подключенного провайдера.

Примером применения является выбор между GitHub OAuth, OIDC и dev-auth режимом через единый интерфейс провайдера аутентификации. UML-диаграмма класса для этого паттерна приведена в приложении А.

Приложение А

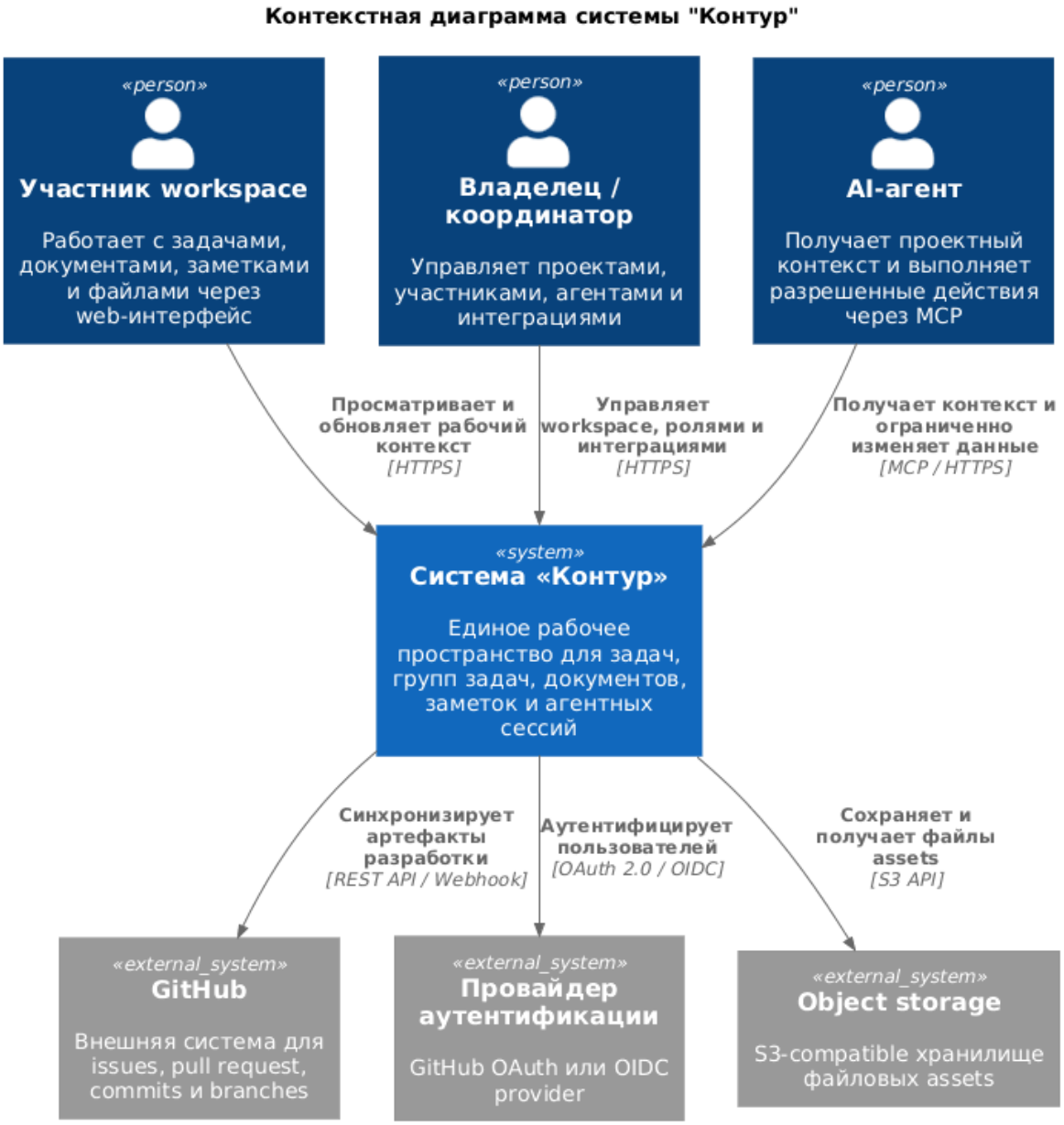

Рисунок А1 – Контекстная диаграмма системы

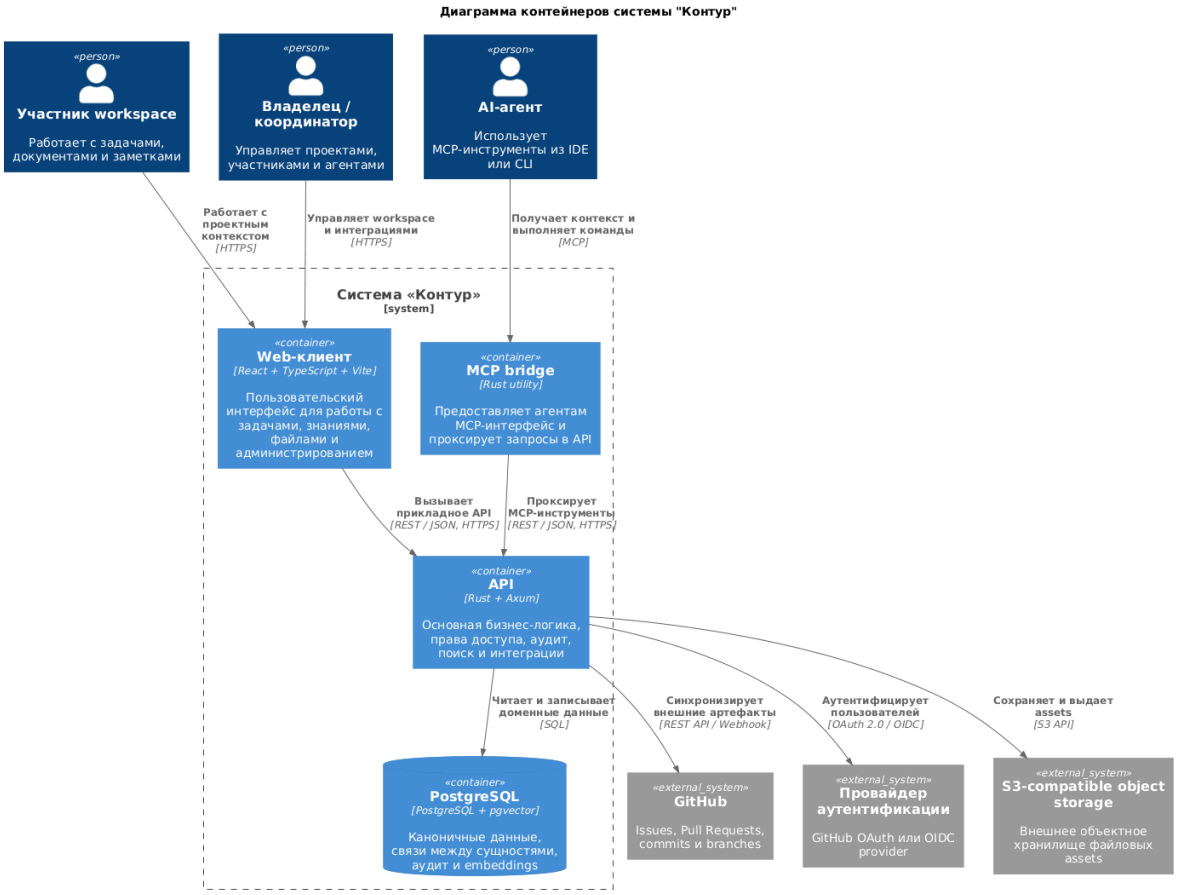

Рисунок А2 – Диаграмма контейнеров системы

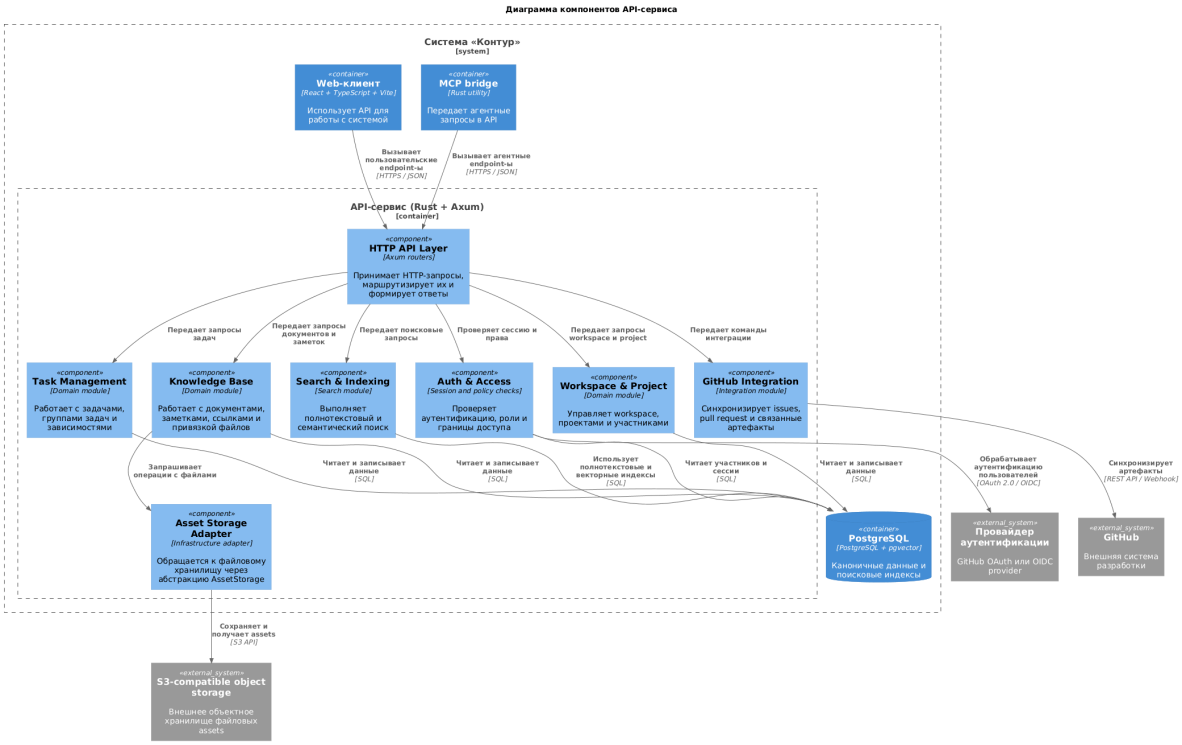

Рисунок А3 – Диаграмма компонентов API-сервиса

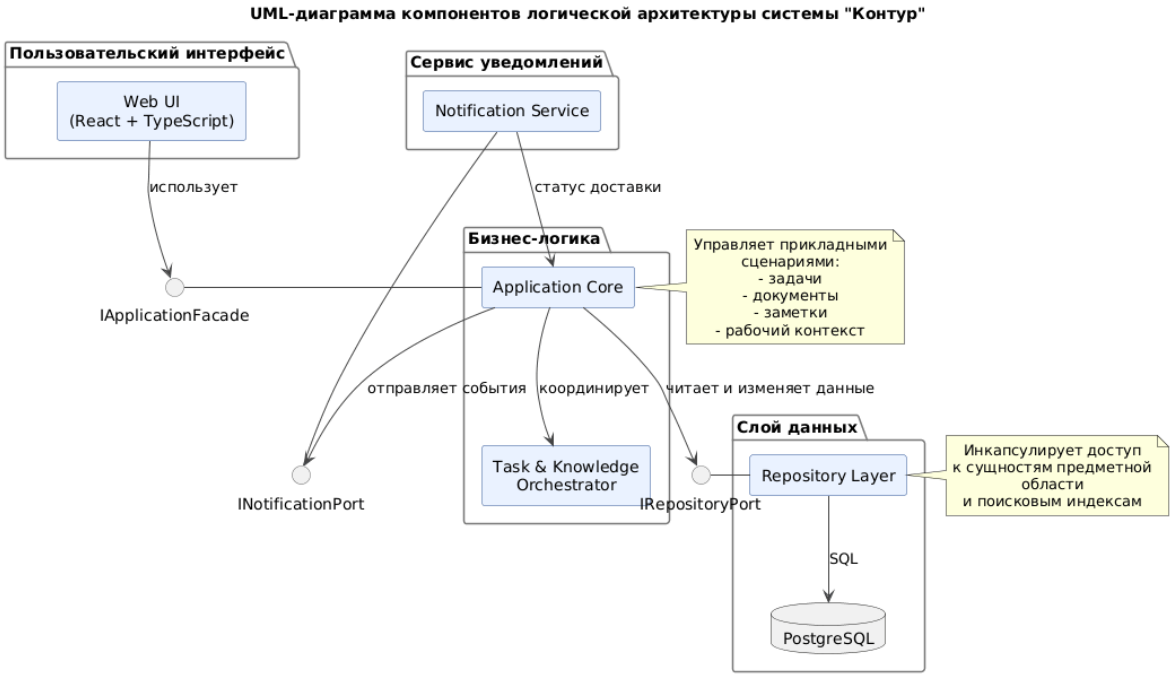

Рисунок А4 – UML-диаграмма компонентов логической архитектуры

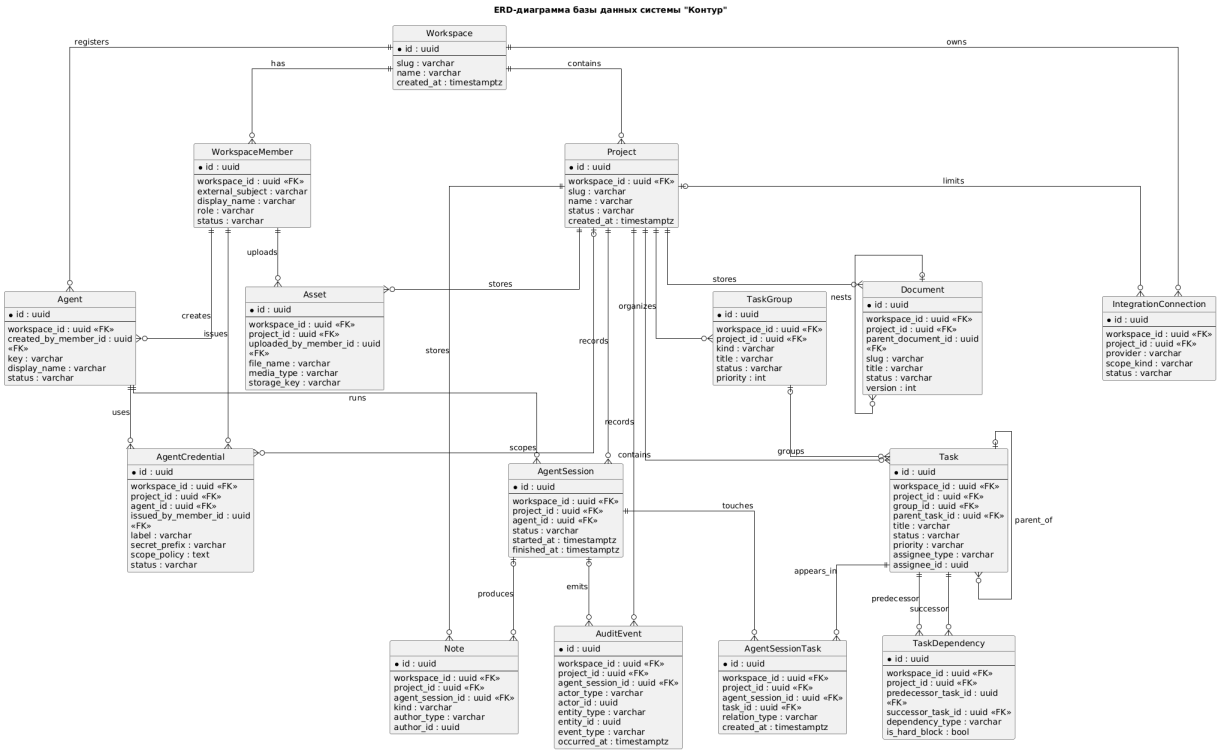

Рисунок А5 – ERD-диаграмма базы данных

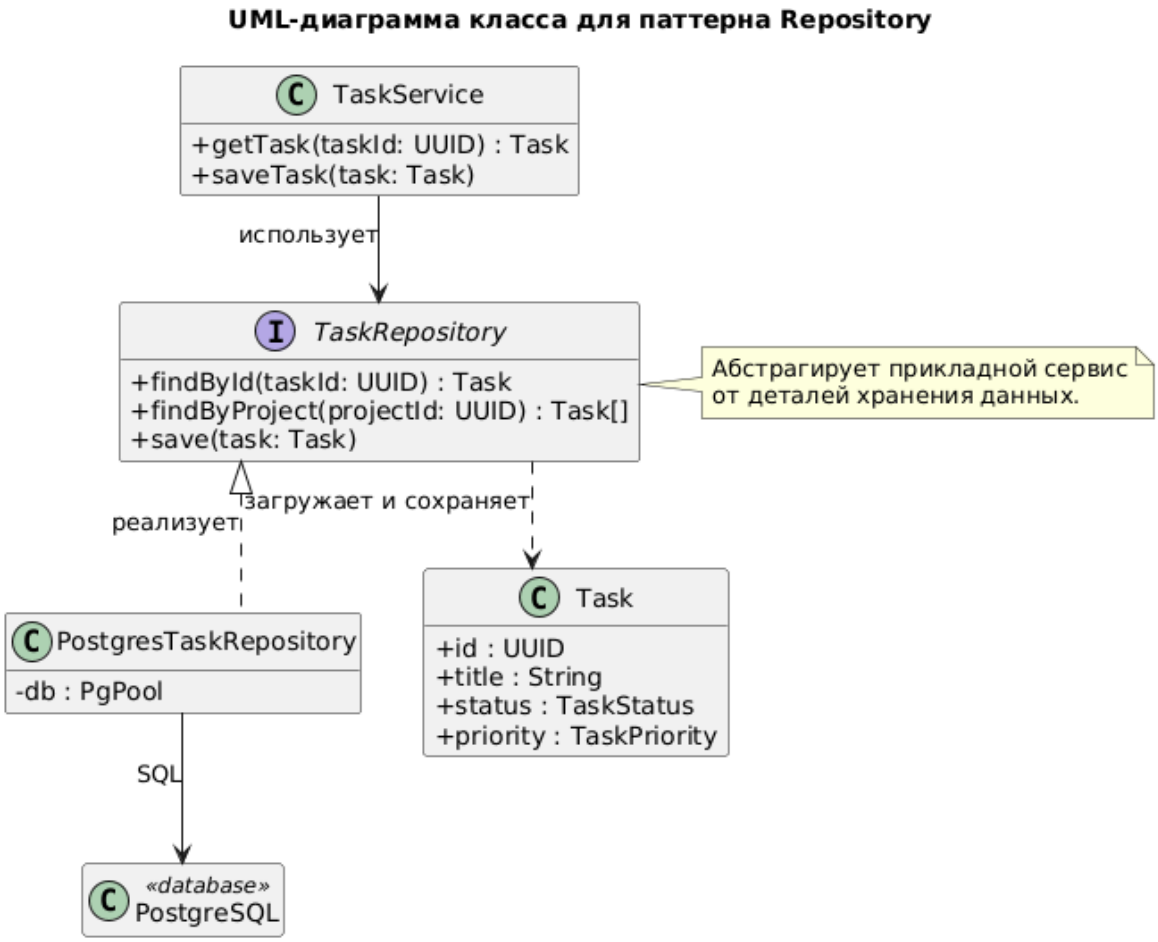

Рисунок А6 – UML-диаграмма класса для паттерна Repository

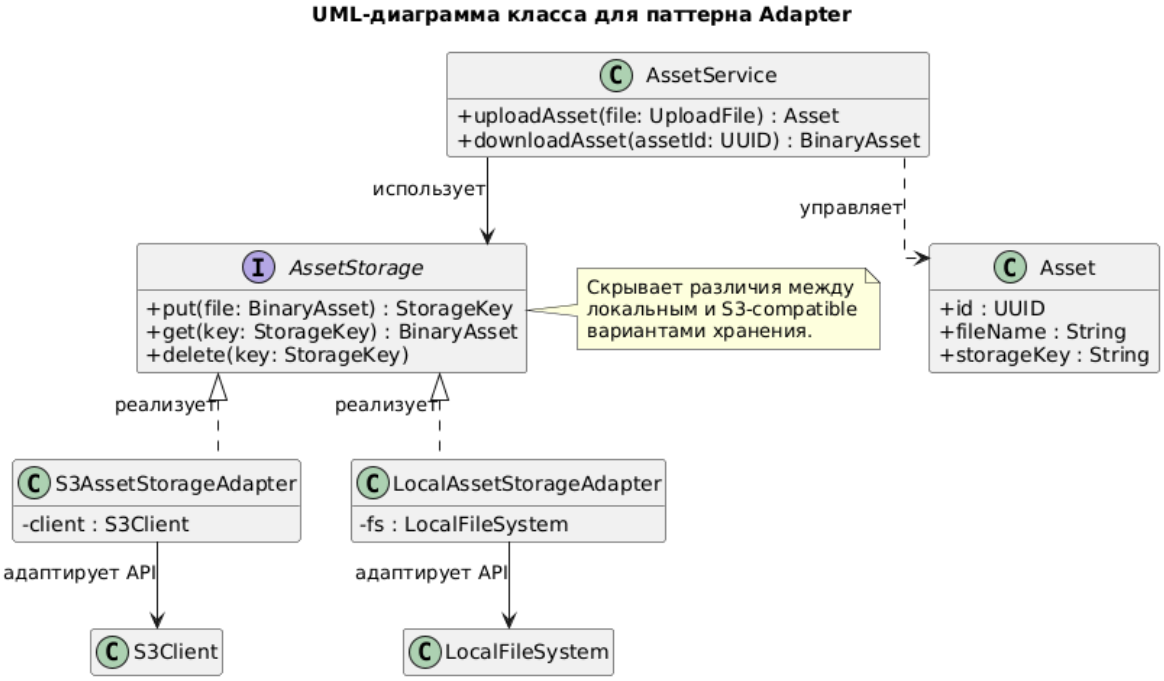

Рисунок А7 – UML-диаграмма класса для паттерна Adapter

Рисунок А8 – UML-диаграмма класса для паттерна Strategy